How Company Search Works — End to End

A complete, plain-English walkthrough of what happens from the moment a user starts a search to the moment a saved company list appears. Written for non-technical reviewers. No code required to follow. Technical file references are listed at the end for engineers.

The journey at a glance

```
1. User types a query OR uploads a brief
        ↓
2. (If uploaded) The document's text is extracted
        ↓
3. A search session is created and saved
        ↓
4. AI translates the request into a fixed checklist of business attributes
        ↓
5. The system fetches matching companies from the database and scores each one
        ↓
6. Companies stream onto a globe + table, live, each with a confidence score
        ↓
7. The user reviews, accepts the ones they want
        ↓
8. Accepted companies are saved as a project and opened in the dashboard
```

Throughout, results appear **progressively** — the user doesn't wait for the whole search to finish; companies pop up one by one as they're found. This is a live stream, not a single "loading… done" request.

Stage 1 — Starting a search

The user has three ways in, from the landing screen:

1. **Type a query** — free text in a search box, e.g. *"Top FMCG distributors in UAE"*.
2. **Upload a brief** — drag-and-drop a PDF, Word, or text file (e.g. a job spec or mandate document).
3. **Import** — bring in an existing list (separate flow, not covered here).

A unique **session ID** is created the moment the screen loads, so everything the user does in this search is grouped together.

Stage 2 — Reading an uploaded document (only if a file is uploaded)

When a brief is uploaded:

- The system **extracts the readable text** from the PDF / Word / text file (up to a size cap). Images and formatting are discarded — only the words matter.
- The extracted text is shown back to the user as a preview, and saved to the session.
- The user can mark the brief **confidential**. If they do, an extra protection step kicks in (see *Stage 4*, confidentiality path) so sensitive names and details are never sent to the external AI.

This extracted text becomes extra context the AI uses alongside (or instead of) a typed query.

Stage 3 — Creating and saving the search session

Before any searching happens, the system records the search so it can be tracked, resumed, and audited:

- A **search session** is saved (the raw query, any uploaded brief text, the confidentiality flag).
- A **draft search record** is created, marked `draft` (not yet a real saved project — just work in progress).
- The user's screen is told the search has started, along with the new search record's ID.

Nothing is "committed" as a real project yet. Drafts that the user abandons simply stay as drafts.

Stage 4 — Understanding the request (the AI "intent" step)

This is the first place AI is used. The system sends the user's text (and any brief context) to a Google Gemini AI model. **The AI's only job is translation** — turning free-form language into a fixed checklist the database understands.

The checklist it fills in:

Attribute	Example	Importance
Primary sector	"Consumer Goods (FMCG)"	**Crucial**
Adjacent sectors	related sectors with similar talent (added automatically, see below)	**Crucial**
Specialism tags	"distribution", "wealth-management"	**Crucial**
Country	"United Arab Emirates"	Nice-to-have
Revenue band	"$250M–$1B"	Nice-to-have
Company size	"1,000–5,000 employees"	Nice-to-have
Public or private	listed / family-owned	Nice-to-have

Three important safeguards built into this step:

- **The AI can only choose from a pre-approved vocabulary.** It cannot invent categories or write anything that reaches the database directly. Any value it returns that isn't on the approved list is discarded. This keeps results consistent and makes the system safe against manipulation.
- **Adjacent sectors are added by the system, not the AI** — once a primary sector is identified, the system looks up a curated list of *related* sectors known to share talent (e.g. Banking → Capital Markets, Insurance). The user never has to ask for these; they widen the net intelligently.
- **Confidentiality path:** if the brief was marked confidential, its raw text is first **summarised into neutral criteria** (sector, geography, size, seniority) *with names and employers stripped out*, and only that neutral summary is sent onward. The sensitive original never leaves the system.

If the AI cannot identify any sector, adjacent sector, or specialism, the search stops cleanly and tells the user *"we couldn't understand a sector for this — try describing an industry or company type."* It does **not**

return random companies. *(This is exactly what happened with the example query "top 10 retails in uae" — the AI step couldn't map it, so the search correctly returned nothing rather than guessing.)*

Stage 5 — Finding and scoring companies (the core matching logic)

The system now looks up real companies in a prepared database table (an "enrichment" dataset containing each company's sector, tags, location, size, revenue, description, and contact details).

How companies are selected — flexible, not strict

The rule that changed recently, and matters most:

- A company is **pulled in if it matches any crucial signal** — its sector, an adjacent sector, or a specialism tag. At least one must match.
- **Nice-to-have criteria never exclude a company.** Country, revenue, size, and public/private status no longer remove anyone from results. They only **raise or lower the company's confidence score**.

> ***Why this matters:*** *Previously the system demanded that a company match every criterion at once — right sector AND right country AND right revenue AND right size. Almost no company ticks every box, so searches often came back **empty** even when strong matches existed. Now the search is forgiving where it should be (the details) and strict where it matters (the sector/specialism).*
>
> *Analogy: instead of rejecting a great candidate over one missing keyword, every reasonable candidate gets an interview — the ones who tick more boxes simply rank higher.*

The system gathers a bounded pool of candidates (it never scans the entire database), then scores each one.

How each company is scored — the confidence percentage

Every candidate gets a **match score from 0% to 100%**, shown as the confidence bar in the results.

The score answers one question: **"Of the things the user asked for, how many did this company satisfy?"** — with the important things weighted more heavily.

Signal matched	Weight (importance)
Primary sector	Highest
Adjacent sector	Medium
Specialism tag	Medium
Country	Small boost
Revenue band	Small boost
Company size	Small boost
Public/private	Tiny boost

Crucial fairness rule: the score only counts what the user actually asked for. If a search never mentioned company size, a size mismatch can't lower the score. So the percentage always means *"how well this fits my specific request"* — not an abstract grade.

Worked examples (search asked for: sector + specialism + country):

- Right sector **and** specialism **and** country → **100%**
- Right sector only → **~57%**
- Adjacent sector + specialism → **~57%**
- Only a specialism tag matched, wrong sector → **~29%** (still shown, clearly lower)

The scoring is **deterministic** — the same company and same query always produce the same score, every time. No randomness, fully predictable and auditable.

Labelling and ordering

Each company is tagged by *why* it matched, with a colour:

Label	Meaning	Colour
Direct	Matched the exact sector requested	Green
Adjacent	Matched a closely related sector	Blue
AI Inferred	Matched on specialism tags but not the sector itself	Amber

Companies are **sorted best-match-first** by confidence score, and each carries a short plain-English reason, e.g. *"Primary sector match (Consumer Goods), in target geography."*

Stage 6 — Saving each found company (safely)

As companies are matched, each is saved to the working list with care:

- **User edits are sacred.** If a company already exists and a person has manually edited a field, the system never overwrites that field.
- **Only gaps are filled.** Empty fields are populated; existing trustworthy data is kept. Lower-quality data never replaces higher-quality data.
- **Every decision is recorded.** The system keeps a per-field history of what value came from where, with what confidence and when — a full audit trail.

Each company is also stamped with its match score, its label (Direct/Adjacent/AI-Inferred), and a one-line reason, so the same information survives if the project is reopened later.

Stage 7 — Live results: the universe view

The user watches results build in real time:

- Matched companies appear on an **interactive globe** (positioned by location) and in a **results table** simultaneously.
- Each row shows the company's details, its **confidence bar**, and its **colour-coded label**.
- While a company is still being processed, a lightweight placeholder ("skeleton") shows so the user sees progress immediately.
- The user can **accept or reject** each company, and can **manually add** a company the search missed.

Optional deeper enrichment (looking up extra revenue figures, headcount, or named executives via AI) exists as a separate, on-demand step — it is **not** part of this core search stream, so the main results stay fast.

Stage 8 — Turning the selection into a saved project

When the user confirms their picks:

- The accepted companies are submitted to the server with the session ID.
- **Ownership is verified** — only companies from this user's own session can be promoted. Users can never pull in another organisation's data.
- The draft search is promoted from `draft` to an **active project**.
- The full company list (plus any executives) is loaded, and the user is taken to the **dashboard**, where the saved companies appear in a full data table for ongoing work and export.

Reopening the project later shows the same companies, scores, labels, and reasons exactly as saved.

Where AI is used vs. plain database work

Step	AI involved?
Extracting text from an uploaded file	No — standard document parsing
Translating the request into the attribute checklist	**Yes** — Google Gemini, *translation only*
Summarising a confidential brief into neutral criteria	**Yes** — Gemini, privacy-protective
Finding & scoring companies	**No** — pure database lookup + fixed scoring math
Optional revenue/executive deep-enrichment	**Yes** — separate, on-demand step

The headline: **the AI only interprets the request. The actual matching and scoring are transparent, rule-based database operations** — predictable, repeatable, and auditable.

Built-in safety guarantees

1. **AI can't break or manipulate the database.** It only picks from a fixed approved vocabulary; anything off-list is thrown away before it reaches the database. No malicious input can ride in through the AI.
2. **User edits are never overwritten** by automation. Manual corrections are permanent unless the user changes them.
3. **Full audit trail** — every automated data change records its source, confidence, and timestamp.
4. **Confidential briefs are protected** — sensitive originals are summarised and stripped before any external AI sees them.
5. **Cross-organisation isolation** — users can only ever save and see their own organisation's companies.
6. **No empty-guessing** — if the request can't be understood, the system says so rather than returning irrelevant companies.

Why this matters — summary for stakeholders

Before	After
Every criterion required at once	Only sector/tag required; the rest refine ranking
Frequent empty results	Relevant results almost always returned

No sense of *how good* a match was	Clear 0–100% confidence on every company
Hidden why a company appeared	Plain-English reason + colour label on each
Over-narrow filters silently killed searches	Narrow filters only lower confidence, never hide good matches
Results appeared all-at-once after a wait	Results stream in live as they're found

Bottom line: the search is now *forgiving where it should be and precise where it matters.* Users get more relevant companies, ranked by a transparent confidence score with a clear reason for each — instead of an empty screen.

Appendix — Technical file map (for engineers)

Stage	Area	Key files
1	Landing / search input	`client/src/pages/Landing/index.tsx`, `panels/SearchPanel.tsx`, `panels/BriefPanel.tsx`
2	File upload + text extraction	`server/routes/registrations/search.ts` (`POST /api/search/upload-brief`), `server/services/briefExtract.ts`
3	Session + draft creation	`server/routes/registrations/search.ts` (`GET /api/search/enhanced-stream`), `server/storage/DatabaseStorage.ts` (`createSearchSession`, `upsertSearchQuery`)
4	Intent extraction (AI)	`server/services/pipeline/enrichmentFilter.ts`, `server/services/llmClient.ts`, `shared/taxonomy.ts` (`adjacentSectorsFor`, `keepInSet`), `server/services/pipeline/briefSummary.ts`
5	Fetch + score	`server/storage/DatabaseStorage.ts` (`queryEnrichedCompanies`), `server/services/pipeline/companyScore.ts`, view `docs/supabase-schema/company_enrichment.sql`
6	Persistence	`server/storage/DatabaseStorage.ts` (`upsertCompanyNonDestructive`), table `hak_companies`
7	Live consumption + render	`client/src/lib/useSearchStream.ts`, `client/src/lib/store.ts`, `client/src/pages/Landing/results/UniverseView.tsx`
8	Accept → project	`client/src/pages/Universe.tsx`, `server/routes/registrations/search.ts` (`POST /api/search/add-to-project`), `client/src/pages/Dashboard.tsx`

Orchestrator: `server/services/pipeline/seedListSearch.ts` (`runSeedListEnhancedStream`).

Live event sequence (server → browser): `search_created` → `intent_extracted` → `adjacent_sector_found` (optional) → `company_found` (placeholder, per company) → `company_enriched` (full data, per company) → `search_complete` (or `no_results`) → `done` .